Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `ref` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 2-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

SimPoint estimates. Ref runs are too long to simulate in full: every value is estimated from up to 30 simulation points of 100 M instructions per run, each after a 500 M-instruction warmup, weighted by SimPoint (`cache_scripts/simpoint.sh`; points and weights in each run directory's `simpoint.txt`). `total_insts` is the whole program's count from qemu.

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin`/`m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	3 831 491 985 262	309 177 145 504	308 680 668 558	495 633 510	843 436	99.8	20.65	0.03	15.638	64.77	4.393	3.597	0.099	0.058
parest	4 088 545 717 646	14 998 028 893	811 009 843	9 923 720 515	4 263 298 536	5.4	0.16	0.49	11.546	63.65	3.604	3.309	0.644	1.338
deepsjeng	2 288 124 945 739	80 869 347 873	19 984 799 357	59 448 342 020	1 436 206 496	24.7	2.92	5.20	9.863	5.80	2.973	7.979	2.799	3.735
leela	2 625 129 460 409	80 675 242 597	2 803 257 082	74 618 676 357	3 253 309 159	3.5	0.38	5.68	6.479	18.20	4.290	0.044	1.767	1.282
namd	2 028 976 157 237	22 071 447 345	22 036 018 281	32 351 019	3 078 044	99.8	3.27	0.00	12.512	25.47	3.309	1.356	11.406	2.721
lbm	1 326 361 633 658	60 519 089 363	60 519 012 839	65 196	11 327	100.0	10.48	0.00	19.643	86.89	0.444	1.753	9.838	1.091
nab	1 734 299 363 064	43 335 663 671	28 093 888 918	13 442 916 323	1 798 858 430	64.8	5.20	1.55	7.987	40.00	1.573	2.822	0.989	1.158
xz_cld	401 325 607 575	6 541 500 109	2 739 318 343	3 420 468 756	381 713 010	41.9	3.38	1.70	11.838	31.57	3.973	1.386	3.233	5.036
xz_cpu2006docs	1 147 585 864 648	6 856 700 239	3 294 943 229	3 244 099 774	317 657 236	48.1	1.47	0.57	9.623	38.70	2.833	2.102	4.705	6.341
xz_combined	625 442 471 499	8 750 690 789	4 136 091 420	4 131 534 419	483 064 950	47.3	2.85	1.32	10.790	22.05	3.985	1.096	3.598	4.955
imagick	4 419 819 220 333	161 009 229 576	985 726 347	112 611 984 461	47 411 518 767	0.6	0.23	5.10	2.175	96.75	3.936	0.030	0.003	0.001
blender	1 882 360 450 846	37 346 363 686	1 777 349 126	31 450 061 421	4 118 953 139	4.8	0.38	3.34	8.221	17.84	2.763	2.064	2.294	5.233
povray	3 547 172 911 376	268 807 318 785	3 408 113 585	248 064 943 702	17 334 261 499	1.3	0.27	13.99	10.321	40.97	7.394	19.422	2.209	1.785
omnetpp	1 183 494 051 463	77 273 945 237	5 553 868	75 458 441 451	1 809 949 918	0.0	0.00	12.75	16.653	27.55	0.085	2.018	3.518	2.901
xalancbmk	1 280 238 262 454	32 984 432 390	514 678 757	30 661 938 211	1 807 815 421	1.6	0.13	4.79	17.454	68.78	10.520	15.819	3.098	5.031
x264_pass1	532 607 517 842	26 480 570 114	17 955 938 896	8 302 056 795	222 574 424	67.8	7.07	3.12	2.782	25.29	0.225	0.935	1.549	1.466
x264_pass2	2 060 055 578 301	88 359 230 989	60 166 511 652	24 461 154 321	3 731 565 017	68.1	6.25	2.37	2.284	31.33	0.359	0.870	1.391	1.418
x264_seek	1 966 389 483 167	85 688 434 921	58 863 030 064	23 684 590 688	3 140 814 169	68.7	6.41	2.41	2.305	27.65	0.320	0.973	1.282	1.409
gcc_pp_O3	224 853 966 645	17 984 502 435	877 592 244	16 879 105 802	227 804 388	4.9	1.23	15.01	9.964	37.45	2.052	5.060	1.067	2.035
gcc_pp_O2	266 929 823 668	21 831 478 671	1 077 827 255	20 499 147 410	254 504 006	4.9	1.25	15.36	10.052	34.58	2.160	5.085	1.120	1.988
gcc_smaller	254 671 373 628	17 265 308 635	931 220 362	16 129 309 979	204 778 295	5.4	1.09	12.67	11.466	43.86	2.779	4.427	0.962	1.871
gcc_ref32_O5	214 010 333 381	16 552 733 395	576 497 719	15 646 044 451	330 191 225	3.5	0.91	14.62	10.160	44.91	3.212	5.142	0.872	1.771
gcc_ref32_O3	318 165 395 272	17 589 262 663	566 719 818	16 818 061 154	204 481 691	3.2	0.68	10.57	11.324	68.76	3.920	6.151	1.006	2.063
mcf	888 079 703 023	7 407 299 183	3 229 620 454	4 144 174 494	33 504 235	43.6	1.40	0.93	22.054	72.44	7.929	7.606	3.444	5.730
perlbench_checkspam	1 309 521 527 045	120 008 085 121	23 137 601 197	93 974 994 957	2 895 488 967	19.3	7.98	14.35	4.538	16.93	0.845	1.777	0.971	0.621
perlbench_diffmail	778 566 201 895	66 166 562 175	3 721 969 520	61 702 382 195	742 210 461	5.6	1.96	15.85	6.043	30.54	2.425	7.792	0.390	1.565
perlbench_splitmail	736 991 491 190	79 002 095 413	7 227 608 106	70 427 685 127	1 346 802 180	9.1	4.16	19.11	9.965	7.65	0.397	4.282	1.510	0.827
bwaves_1	497 063 723 317	2 989 609 538	2 989 191 752	351 316	66 470	100.0	2.51	0.00	43.266	24.07	3.940	0.198	4.543	5.106
bwaves_2	784 691 829 793	4 892 060 128	4 891 557 364	404 109	98 655	100.0	2.59	0.00	42.951	24.39	3.803	0.230	7.461	5.187
bwaves_3	611 703 570 107	3 688 669 131	3 688 117 546	439 481	112 104	100.0	2.51	0.00	42.881	24.27	4.736	0.174	5.759	4.510
bwaves_4	745 048 195 941	4 606 093 801	4 605 716 550	319 055	58 197	100.0	2.56	0.00	42.814	24.15	3.885	0.165	4.857	5.803
exchange2	2 120 109 818 190	67 506 596 157	51 688 048 432	8 987 153 068	6 831 394 657	76.6	5.23	0.85	1.834	0.85	1.622	2.782	8.119	5.404
fotonik3d	2 457 798 604 788	19 150 475 290	18 811 665 225	309 878 797	28 931 268	98.2	3.36	0.03	16.671	41.15	6.136	0.171	0.106	3.443
roms	2 733 593 366 577	17 773 282 831	16 141 154 959	967 833 899	664 293 973	90.8	4.37	0.07	21.552	38.82	4.011	1.248	2.843	3.810
wrf	4 131 413 910 638	19 044 564 711	5 282 083 873	12 615 417 654	1 147 063 184	27.7	0.91	0.61	17.877	21.73	9.005	5.948	0.812	1.723
cam4	2 349 180 609 851	44 573 582 631	21 602 368 027	21 690 037 915	1 281 176 689	48.5	3.14	1.85	11.801	50.53	3.178	3.082	1.110	2.106
Total	58 391 814 127 468	1 959 776 643 990	**767 822 470 568**	1 084 245 719 802	107 708 453 623	39.2	3.78	3.71	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `ref` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, fully associative L1D/L2 (FALRU) · gem5 `e0a12aa4e3+dirty`

SimPoint estimates. Ref runs are too long to simulate in full: every value is estimated from up to 30 simulation points of 100 M instructions per run, each after a 500 M-instruction warmup, weighted by SimPoint (`cache_scripts/simpoint.sh`; points and weights in each run directory's `simpoint.txt`). `total_insts` is the whole program's count from qemu.

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin`/`m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	3 831 491 985 262	309 177 145 504	308 680 668 558	495 633 510	843 436	99.8	20.65	0.03	15.369	61.01	4.994	4.972	0.033	0.049
parest	4 088 545 717 646	14 998 028 893	811 009 843	9 923 720 515	4 263 298 536	5.4	0.16	0.49	9.783	75.05	1.182	1.282	0.311	0.652
deepsjeng	2 288 124 945 739	80 869 347 873	19 984 799 357	59 448 342 020	1 436 206 496	24.7	2.92	5.20	7.893	5.04	1.590	7.478	1.850	3.260
leela	2 625 129 460 409	80 675 242 597	2 803 257 082	74 618 676 357	3 253 309 159	3.5	0.38	5.68	4.021	28.13	0.060	0.001	0.612	0.546
namd	2 028 976 157 237	22 071 447 345	22 036 018 281	32 351 019	3 078 044	99.8	3.27	0.00	11.541	27.21	2.408	1.286	11.304	2.547
lbm	1 326 361 633 658	60 519 089 363	60 519 012 839	65 196	11 327	100.0	10.48	0.00	17.101	99.98	0.001	0.023	9.464	0.966
nab	1 734 299 363 064	43 335 663 671	28 093 888 918	13 442 916 323	1 798 858 430	64.8	5.20	1.55	3.906	82.17	0.082	0.254	0.558	0.739
xz_cld	401 325 607 575	6 541 500 109	2 739 318 343	3 420 468 756	381 713 010	41.9	3.38	1.70	10.093	36.73	0.497	0.266	1.162	0.383
xz_cpu2006docs	1 147 585 864 648	6 856 700 239	3 294 943 229	3 244 099 774	317 657 236	48.1	1.47	0.57	9.296	39.57	1.371	0.971	1.921	5.393
xz_combined	625 442 471 499	8 750 690 789	4 136 091 420	4 131 534 419	483 064 950	47.3	2.85	1.32	9.690	24.09	0.765	0.472	1.460	1.524
imagick	4 419 819 220 333	161 009 229 576	985 726 347	112 611 984 461	47 411 518 767	0.6	0.23	5.10	2.224	100.00	3.943	0.011	0.000	0.001
blender	1 882 360 450 846	37 346 363 686	1 777 349 126	31 450 061 421	4 118 953 139	4.8	0.38	3.34	8.009	17.55	2.595	1.355	2.086	4.903
povray	3 547 172 911 376	268 807 318 785	3 408 113 585	248 064 943 702	17 334 261 499	1.3	0.27	13.99	8.488	50.78	7.255	18.469	1.786	0.862
omnetpp	1 183 494 051 463	77 273 945 237	5 553 868	75 458 441 451	1 809 949 918	0.0	0.00	12.75	16.238	27.41	0.076	1.362	2.133	3.269
xalancbmk	1 280 238 262 454	32 984 432 390	514 678 757	30 661 938 211	1 807 815 421	1.6	0.13	4.79	16.810	70.85	11.161	22.223	2.604	4.370
x264_pass1	532 607 517 842	26 480 570 114	17 955 938 896	8 302 056 795	222 574 424	67.8	7.07	3.12	2.294	24.98	0.139	1.008	1.072	1.218
x264_pass2	2 060 055 578 301	88 359 230 989	60 166 511 652	24 461 154 321	3 731 565 017	68.1	6.25	2.37	1.827	36.21	0.105	0.711	0.951	1.053
x264_seek	1 966 389 483 167	85 688 434 921	58 863 030 064	23 684 590 688	3 140 814 169	68.7	6.41	2.41	1.830	32.07	0.119	0.815	1.023	1.088
gcc_pp_O3	224 853 966 645	17 984 502 435	877 592 244	16 879 105 802	227 804 388	4.9	1.23	15.01	8.026	43.50	0.782	4.614	0.514	1.400
gcc_pp_O2	266 929 823 668	21 831 478 671	1 077 827 255	20 499 147 410	254 504 006	4.9	1.25	15.36	8.271	39.71	0.767	4.654	0.505	1.349
gcc_smaller	254 671 373 628	17 265 308 635	931 220 362	16 129 309 979	204 778 295	5.4	1.09	12.67	9.911	51.24	1.351	3.234	0.503	1.176
gcc_ref32_O5	214 010 333 381	16 552 733 395	576 497 719	15 646 044 451	330 191 225	3.5	0.91	14.62	7.924	54.04	0.974	3.449	0.348	0.838
gcc_ref32_O3	318 165 395 272	17 589 262 663	566 719 818	16 818 061 154	204 481 691	3.2	0.68	10.57	10.270	74.31	1.782	5.263	0.456	1.234
mcf	888 079 703 023	7 407 299 183	3 229 620 454	4 144 174 494	33 504 235	43.6	1.40	0.93	21.772	72.93	2.592	6.127	2.346	5.144
perlbench_checkspam	1 309 521 527 045	120 008 085 121	23 137 601 197	93 974 994 957	2 895 488 967	19.3	7.98	14.35	1.727	35.71	0.089	0.309	0.115	0.348
perlbench_diffmail	778 566 201 895	66 166 562 175	3 721 969 520	61 702 382 195	742 210 461	5.6	1.96	15.85	2.753	51.55	0.165	2.311	0.110	0.374
perlbench_splitmail	736 991 491 190	79 002 095 413	7 227 608 106	70 427 685 127	1 346 802 180	9.1	4.16	19.11	10.141	6.97	0.026	10.373	0.022	1.297
bwaves_1	497 063 723 317	2 989 609 538	2 989 191 752	351 316	66 470	100.0	2.51	0.00	7.334	97.66	0.752	0.001	3.888	5.235
bwaves_2	784 691 829 793	4 892 060 128	4 891 557 364	404 109	98 655	100.0	2.59	0.00	7.293	97.59	0.735	0.001	6.866	5.003
bwaves_3	611 703 570 107	3 688 669 131	3 688 117 546	439 481	112 104	100.0	2.51	0.00	7.302	97.86	0.812	0.001	5.227	4.594
bwaves_4	745 048 195 941	4 606 093 801	4 605 716 550	319 055	58 197	100.0	2.56	0.00	7.250	98.43	0.576	0.001	4.251	5.554
exchange2	2 120 109 818 190	67 506 596 157	51 688 048 432	8 987 153 068	6 831 394 657	76.6	5.23	0.85	1.128	1.41	1.289	1.649	2.669	1.934
fotonik3d	2 457 798 604 788	19 150 475 290	18 811 665 225	309 878 797	28 931 268	98.2	3.36	0.03	8.228	83.12	2.056	0.013	0.018	3.311
roms	2 733 593 366 577	17 773 282 831	16 141 154 959	967 833 899	664 293 973	90.8	4.37	0.07	10.864	67.88	1.549	0.723	2.830	2.574
wrf	4 131 413 910 638	19 044 571 895	5 282 083 873	12 615 392 001	1 147 096 021	27.7	0.91	0.61	5.679	62.89	5.100	3.730	0.457	1.381
cam4	2 349 180 609 851	44 573 582 631	21 602 368 027	21 690 037 915	1 281 176 689	48.5	3.14	1.85	10.387	56.58	1.796	2.109	0.638	1.626
Total	58 391 814 127 468	1 959 776 651 174	**767 822 470 568**	1 084 245 694 149	107 708 486 460	39.2	3.78	3.71	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `ref` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, direct-mapped L1D/L2 · gem5 `e0a12aa4e3+dirty`

SimPoint estimates. Ref runs are too long to simulate in full: every value is estimated from up to 30 simulation points of 100 M instructions per run, each after a 500 M-instruction warmup, weighted by SimPoint (`cache_scripts/simpoint.sh`; points and weights in each run directory's `simpoint.txt`). `total_insts` is the whole program's count from qemu.

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin`/`m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	3 831 491 985 262	309 177 145 504	308 680 668 558	495 633 510	843 436	99.8	20.65	0.03	16.944	55.00	6.469	5.005	1.103	0.122
parest	4 088 545 717 646	14 998 028 893	811 009 843	9 923 720 515	4 263 298 536	5.4	0.16	0.49	15.193	51.66	10.387	5.801	1.707	2.971
deepsjeng	2 288 124 945 739	80 869 347 873	19 984 799 357	59 448 342 020	1 436 206 496	24.7	2.92	5.20	13.614	14.36	6.436	9.665	4.229	4.998
leela	2 625 129 460 409	80 675 242 597	2 803 257 082	74 618 676 357	3 253 309 159	3.5	0.38	5.68	7.977	17.91	7.042	0.276	3.411	2.962
namd	2 028 976 157 237	22 071 447 345	22 036 018 281	32 351 019	3 078 044	99.8	3.27	0.00	14.973	32.32	6.995	2.521	11.621	3.941
lbm	1 326 361 633 658	60 519 089 363	60 519 012 839	65 196	11 327	100.0	10.48	0.00	22.888	78.54	1.677	5.035	9.795	1.587
nab	1 734 299 363 064	43 335 663 671	28 093 888 918	13 442 916 323	1 798 858 430	64.8	5.20	1.55	13.737	31.34	3.930	2.846	2.701	2.526
xz_cld	401 325 607 575	6 541 500 109	2 739 318 343	3 420 468 756	381 713 010	41.9	3.38	1.70	15.160	34.07	3.619	2.695	4.784	5.664
xz_cpu2006docs	1 147 585 864 648	6 856 700 239	3 294 943 229	3 244 099 774	317 657 236	48.1	1.47	0.57	11.354	41.33	4.715	3.053	6.603	6.834
xz_combined	625 442 471 499	8 750 690 789	4 136 091 420	4 131 534 419	483 064 950	47.3	2.85	1.32	14.394	28.18	5.201	3.884	5.497	6.597
imagick	4 419 819 220 333	161 009 229 576	985 726 347	112 611 984 461	47 411 518 767	0.6	0.23	5.10	2.761	68.78	6.340	1.402	0.142	0.151
blender	1 882 360 450 846	37 346 363 686	1 777 349 126	31 450 061 421	4 118 953 139	4.8	0.38	3.34	10.333	31.65	7.496	3.363	3.123	8.373
povray	3 547 172 911 376	268 807 318 785	3 408 113 585	248 064 943 702	17 334 261 499	1.3	0.27	13.99	14.403	37.67	9.214	22.195	5.874	3.607
omnetpp	1 183 494 051 463	77 273 945 237	5 553 868	75 458 441 451	1 809 949 918	0.0	0.00	12.75	19.327	37.69	12.851	2.128	5.539	4.066
xalancbmk	1 280 238 262 454	32 984 432 390	514 678 757	30 661 938 211	1 807 815 421	1.6	0.13	4.79	18.918	68.27	13.795	17.840	6.628	5.997
x264_pass1	532 607 517 842	26 480 570 114	17 955 938 896	8 302 056 795	222 574 424	67.8	7.07	3.12	4.658	24.17	0.692	1.383	1.841	2.341
x264_pass2	2 060 055 578 301	88 359 230 989	60 166 511 652	24 461 154 321	3 731 565 017	68.1	6.25	2.37	4.156	28.76	0.776	1.107	1.796	1.996
x264_seek	1 966 389 483 167	85 688 434 921	58 863 030 064	23 684 590 688	3 140 814 169	68.7	6.41	2.41	4.255	23.46	0.918	1.374	1.915	2.281
gcc_pp_O3	224 853 966 645	17 984 502 435	877 592 244	16 879 105 802	227 804 388	4.9	1.23	15.01	13.486	38.93	7.066	8.261	2.801	4.148
gcc_pp_O2	266 929 823 668	21 831 478 671	1 077 827 255	20 499 147 410	254 504 006	4.9	1.25	15.36	13.503	37.38	6.928	8.628	2.728	4.147
gcc_smaller	254 671 373 628	17 265 308 635	931 220 362	16 129 309 979	204 778 295	5.4	1.09	12.67	14.566	45.12	6.218	7.166	2.625	4.085
gcc_ref32_O5	214 010 333 381	16 552 733 395	576 497 719	15 646 044 451	330 191 225	3.5	0.91	14.62	13.627	44.76	7.016	8.714	2.609	3.803
gcc_ref32_O3	318 165 395 272	17 589 262 663	566 719 818	16 818 061 154	204 481 691	3.2	0.68	10.57	13.935	62.89	9.317	10.742	2.875	4.360
mcf	888 079 703 023	7 407 299 183	3 229 620 454	4 144 174 494	33 504 235	43.6	1.40	0.93	24.459	69.97	11.703	9.976	5.265	6.335
perlbench_checkspam	1 309 521 527 045	120 008 085 121	23 137 601 197	93 974 994 957	2 895 488 967	19.3	7.98	14.35	6.932	19.65	2.207	3.552	1.883	1.588
perlbench_diffmail	778 566 201 895	66 166 562 175	3 721 969 520	61 702 382 195	742 210 461	5.6	1.96	15.85	8.955	29.10	2.244	12.126	3.302	3.695
perlbench_splitmail	736 991 491 190	79 002 095 413	7 227 608 106	70 427 685 127	1 346 802 180	9.1	4.16	19.11	13.581	14.31	1.382	8.342	3.438	5.751
bwaves_1	497 063 723 317	2 989 609 538	2 989 191 752	351 316	66 470	100.0	2.51	0.00	44.408	90.83	7.116	1.067	4.956	6.224
bwaves_2	784 691 829 793	4 892 060 128	4 891 557 364	404 109	98 655	100.0	2.59	0.00	44.035	91.21	7.032	1.143	7.902	5.939
bwaves_3	611 703 570 107	3 688 669 131	3 688 117 546	439 481	112 104	100.0	2.51	0.00	45.972	41.97	7.161	1.062	6.557	5.013
bwaves_4	745 048 195 941	4 606 093 801	4 605 716 550	319 055	58 197	100.0	2.56	0.00	44.008	44.00	7.084	1.214	5.106	6.279
exchange2	2 120 109 818 190	67 506 596 157	51 688 048 432	8 987 153 068	6 831 394 657	76.6	5.23	0.85	3.113	0.55	1.880	2.009	2.541	2.789
fotonik3d	2 457 798 604 788	19 150 475 290	18 811 665 225	309 878 797	28 931 268	98.2	3.36	0.03	21.997	49.28	9.791	0.495	0.713	3.454
roms	2 733 593 366 577	17 773 282 831	16 141 154 959	967 833 899	664 293 973	90.8	4.37	0.07	30.332	40.55	6.830	3.248	4.703	7.684
wrf	4 131 413 910 638	19 044 546 472	5 282 083 873	12 615 396 106	1 147 066 493	27.7	0.91	0.61	26.093	29.03	13.663	8.197	5.805	2.760
cam4	2 349 180 609 851	44 573 582 631	21 602 368 027	21 690 037 915	1 281 176 689	48.5	3.14	1.85	15.118	47.00	5.766	4.622	2.023	3.234
Total	58 391 814 127 468	1 959 776 625 751	**767 822 470 568**	1 084 245 698 254	107 708 456 932	39.2	3.78	3.71	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `ref` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 4-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

SimPoint estimates. Ref runs are too long to simulate in full: every value is estimated from up to 30 simulation points of 100 M instructions per run, each after a 500 M-instruction warmup, weighted by SimPoint (`cache_scripts/simpoint.sh`; points and weights in each run directory's `simpoint.txt`). `total_insts` is the whole program's count from qemu.

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin`/`m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	3 831 491 985 262	309 177 145 504	308 680 668 558	495 633 510	843 436	99.8	20.65	0.03	15.631	64.99	5.185	4.296	0.034	0.051
parest	4 088 545 717 646	14 998 028 893	811 009 843	9 923 720 515	4 263 298 536	5.4	0.16	0.49	10.383	70.85	2.355	1.667	0.381	0.855
deepsjeng	2 288 124 945 739	80 869 347 873	19 984 799 357	59 448 342 020	1 436 206 496	24.7	2.92	5.20	8.811	6.35	2.098	7.453	2.119	3.329
leela	2 625 129 460 409	80 675 242 597	2 803 257 082	74 618 676 357	3 253 309 159	3.5	0.38	5.68	4.941	23.48	0.510	0.002	1.163	0.920
namd	2 028 976 157 237	22 071 447 345	22 036 018 281	32 351 019	3 078 044	99.8	3.27	0.00	11.577	27.50	2.404	1.259	11.362	2.686
lbm	1 326 361 633 658	60 519 089 363	60 519 012 839	65 196	11 327	100.0	10.48	0.00	17.634	97.01	0.014	0.115	9.507	1.048
nab	1 734 299 363 064	43 335 663 671	28 093 888 918	13 442 916 323	1 798 858 430	64.8	5.20	1.55	5.400	59.18	0.130	0.688	0.634	0.755
xz_cld	401 325 607 575	6 541 500 109	2 739 318 343	3 420 468 756	381 713 010	41.9	3.38	1.70	10.656	35.09	2.045	1.513	1.906	3.096
xz_cpu2006docs	1 147 585 864 648	6 856 700 239	3 294 943 229	3 244 099 774	317 657 236	48.1	1.47	0.57	9.404	39.60	2.196	1.905	3.190	5.011
xz_combined	625 442 471 499	8 750 690 789	4 136 091 420	4 131 534 419	483 064 950	47.3	2.85	1.32	9.724	24.47	1.976	1.297	2.416	2.495
imagick	4 419 819 220 333	161 009 229 576	985 726 347	112 611 984 461	47 411 518 767	0.6	0.23	5.10	2.178	96.60	3.929	0.032	0.000	0.001
blender	1 882 360 450 846	37 346 363 686	1 777 349 126	31 450 061 421	4 118 953 139	4.8	0.38	3.34	8.000	18.31	2.666	1.425	2.180	4.787
povray	3 547 172 911 376	268 807 318 785	3 408 113 585	248 064 943 702	17 334 261 499	1.3	0.27	13.99	9.157	45.94	7.206	18.522	1.705	1.066
omnetpp	1 183 494 051 463	77 273 945 237	5 553 868	75 458 441 451	1 809 949 918	0.0	0.00	12.75	16.311	28.08	0.076	1.496	2.610	3.051
xalancbmk	1 280 238 262 454	32 984 432 390	514 678 757	30 661 938 211	1 807 815 421	1.6	0.13	4.79	17.119	70.02	11.801	22.711	2.901	4.521
x264_pass1	532 607 517 842	26 480 570 114	17 955 938 896	8 302 056 795	222 574 424	67.8	7.07	3.12	2.421	28.58	0.133	1.004	1.167	1.363
x264_pass2	2 060 055 578 301	88 359 230 989	60 166 511 652	24 461 154 321	3 731 565 017	68.1	6.25	2.37	1.949	35.53	0.185	0.776	0.994	1.078
x264_seek	1 966 389 483 167	85 688 434 921	58 863 030 064	23 684 590 688	3 140 814 169	68.7	6.41	2.41	1.962	31.56	0.156	0.908	1.035	1.135
gcc_pp_O3	224 853 966 645	17 984 502 435	877 592 244	16 879 105 802	227 804 388	4.9	1.23	15.01	8.737	41.84	1.298	4.768	0.646	1.534
gcc_pp_O2	266 929 823 668	21 831 478 671	1 077 827 255	20 499 147 410	254 504 006	4.9	1.25	15.36	8.893	38.46	1.190	4.312	0.709	1.464
gcc_smaller	254 671 373 628	17 265 308 635	931 220 362	16 129 309 979	204 778 295	5.4	1.09	12.67	10.479	47.59	2.045	3.709	0.567	1.282
gcc_ref32_O5	214 010 333 381	16 552 733 395	576 497 719	15 646 044 451	330 191 225	3.5	0.91	14.62	8.862	50.61	1.679	4.070	0.453	1.108
gcc_ref32_O3	318 165 395 272	17 589 262 663	566 719 818	16 818 061 154	204 481 691	3.2	0.68	10.57	10.652	72.66	2.477	5.119	0.575	1.363
mcf	888 079 703 023	7 407 299 183	3 229 620 454	4 144 174 494	33 504 235	43.6	1.40	0.93	21.529	74.18	6.088	6.445	2.546	5.330
perlbench_checkspam	1 309 521 527 045	120 008 085 121	23 137 601 197	93 974 994 957	2 895 488 967	19.3	7.98	14.35	2.496	28.16	0.097	0.427	0.266	0.472
perlbench_diffmail	778 566 201 895	66 166 562 175	3 721 969 520	61 702 382 195	742 210 461	5.6	1.96	15.85	4.512	37.92	0.127	4.559	0.161	0.780
perlbench_splitmail	736 991 491 190	79 002 095 413	7 227 608 106	70 427 685 127	1 346 802 180	9.1	4.16	19.11	8.879	8.50	0.033	7.272	0.206	0.860
bwaves_1	497 063 723 317	2 989 609 538	2 989 191 752	351 316	66 470	100.0	2.51	0.00	44.121	23.63	6.223	0.104	3.836	5.057
bwaves_2	784 691 829 793	4 892 060 128	4 891 557 364	404 109	98 655	100.0	2.59	0.00	43.897	23.88	6.018	0.121	6.867	4.802
bwaves_3	611 703 570 107	3 688 669 131	3 688 117 546	439 481	112 104	100.0	2.51	0.00	43.191	24.11	6.111	0.001	5.174	4.519
bwaves_4	745 048 195 941	4 606 093 801	4 605 716 550	319 055	58 197	100.0	2.56	0.00	43.283	23.87	5.840	0.001	4.176	5.478
exchange2	2 120 109 818 190	67 506 596 157	51 688 048 432	8 987 153 068	6 831 394 657	76.6	5.23	0.85	1.667	0.94	1.409	2.236	6.668	2.222
fotonik3d	2 457 798 604 788	19 150 475 290	18 811 665 225	309 878 797	28 931 268	98.2	3.36	0.03	10.254	66.95	4.031	0.045	0.018	3.311
roms	2 733 593 366 577	17 773 282 831	16 141 154 959	967 833 899	664 293 973	90.8	4.37	0.07	15.129	53.37	2.310	0.521	2.831	2.643
wrf	4 131 413 910 638	19 044 552 577	5 282 083 873	12 615 402 211	1 147 066 493	27.7	0.91	0.61	11.267	33.77	6.626	4.405	0.562	1.570
cam4	2 349 180 609 851	44 573 582 631	21 602 368 027	21 690 037 915	1 281 176 689	48.5	3.14	1.85	10.694	55.53	2.039	2.355	0.692	1.622
Total	58 391 814 127 468	1 959 776 631 856	**767 822 470 568**	1 084 245 704 359	107 708 456 932	39.2	3.78	3.71	0.000	—	—	—	—	—

Improving CPU Efficiency for Data-Intensive Applications through Cache-Aware Spill Optimization

Register Spill Behaviour and Cache Hit/Miss Rates on RISC-V SPEC CPU2017

Lütfullah Burak Kaya
Ozyegin University
burak.kaya.50711@ozu.edu.tr

Ismail Akturk
Ozyegin University
ismail.akturk@ozyegin.edu.tr

4 November 2026

gem5 TimingSimpleCPU · workload profile `ref` · L1D 4 KiB, L1I 32 KiB, L2 32 KiB, L1D 8-way, L2 8-way · gem5 `e0a12aa4e3+dirty`

SimPoint estimates. Ref runs are too long to simulate in full: every value is estimated from up to 30 simulation points of 100 M instructions per run, each after a 500 M-instruction warmup, weighted by SimPoint (`cache_scripts/simpoint.sh`; points and weights in each run directory's `simpoint.txt`). `total_insts` is the whole program's count from qemu.

Definitions (metric schema 9). Every stack store issued inside the region of interest, and every later load that reads it back (the load's address lies in the bytes the store wrote; widths may differ), belongs to exactly one class. `spill`: register-allocator spill code, identified by GCC itself (`-dP` RTL dumps of the same build, memory operands in the LRA spill area). `csave`: callee-saved register save/restore, return address included, identified from the binary's CFI (`.eh_frame`). `other_stack`: every remaining SP/FP-relative stack store and reload (stack-passed arguments, memory-resident locals). `stack_*` is the sum of the three. For each class an *event* is one (store, first reload) pair, so `event` $\leq$ `store` and `load` $\geq$ `event`. All counters cover only the region delimited by `m5_work_begin`/`m5_work_end`: the gem5 statistics are dumped and reset at those boundaries, so the cache columns span the same window, and the cache tags carry the class, so per-class hit/miss rates come straight from gem5. *Spill %* and *Csave %* are the class's store+reload instructions over `ROI_insts`; *spill/stack %* is the share of stack events that are real spills.

Benchmark	ROI_insts	stack_event	**spill_event**	csave_event	other_event	spill/stack %	Spill %	Csave %	L1D m %	L2 m %	sp_load m %	sp_store m %	cs_load m %	cs_store m %
cactuBSSN	3 831 491 985 262	309 177 145 504	308 680 668 558	495 633 510	843 436	99.8	20.65	0.03	15.859	64.24	5.459	4.615	0.034	0.048
parest	4 088 545 717 646	14 998 028 893	811 009 843	9 923 720 515	4 263 298 536	5.4	0.16	0.49	9.942	73.98	1.371	1.333	0.337	0.692
deepsjeng	2 288 124 945 739	80 869 347 873	19 984 799 357	59 448 342 020	1 436 206 496	24.7	2.92	5.20	8.513	6.50	1.845	7.297	1.884	3.329
leela	2 625 129 460 409	80 675 242 597	2 803 257 082	74 618 676 357	3 253 309 159	3.5	0.38	5.68	4.197	27.29	0.062	0.001	0.700	0.492
namd	2 028 976 157 237	22 071 447 345	22 036 018 281	32 351 019	3 078 044	99.8	3.27	0.00	11.532	27.58	2.428	1.264	11.316	2.582
lbm	1 326 361 633 658	60 519 089 363	60 519 012 839	65 196	11 327	100.0	10.48	0.00	17.448	97.87	0.001	0.101	9.464	1.379
nab	1 734 299 363 064	43 335 663 671	28 093 888 918	13 442 916 323	1 798 858 430	64.8	5.20	1.55	4.129	77.32	0.093	0.264	0.583	0.729
xz_cld	401 325 607 575	6 541 500 109	2 739 318 343	3 420 468 756	381 713 010	41.9	3.38	1.70	10.242	36.46	1.022	0.482	1.318	0.905
xz_cpu2006docs	1 147 585 864 648	6 856 700 239	3 294 943 229	3 244 099 774	317 657 236	48.1	1.47	0.57	9.308	40.00	1.546	1.369	2.401	4.938
xz_combined	625 442 471 499	8 750 690 789	4 136 091 420	4 131 534 419	483 064 950	47.3	2.85	1.32	9.531	24.91	1.133	0.713	1.739	1.114
imagick	4 419 819 220 333	161 009 229 576	985 726 347	112 611 984 461	47 411 518 767	0.6	0.23	5.10	2.188	100.00	3.595	0.013	0.000	0.001
blender	1 882 360 450 846	37 346 363 686	1 777 349 126	31 450 061 421	4 118 953 139	4.8	0.38	3.34	7.967	18.37	2.591	1.313	2.115	4.705
povray	3 547 172 911 376	268 807 318 785	3 408 113 585	248 064 943 702	17 334 261 499	1.3	0.27	13.99	8.454	49.55	7.247	18.271	1.500	0.900
omnetpp	1 183 494 051 463	77 273 945 237	5 553 868	75 458 441 451	1 809 949 918	0.0	0.00	12.75	16.271	28.12	0.076	1.189	2.264	3.049
xalancbmk	1 280 238 262 454	32 984 432 390	514 678 757	30 661 938 211	1 807 815 421	1.6	0.13	4.79	16.843	71.10	11.165	20.103	2.607	4.267
x264_pass1	532 607 517 842	26 480 570 114	17 955 938 896	8 302 056 795	222 574 424	67.8	7.07	3.12	2.365	29.18	0.099	0.991	1.111	1.284
x264_pass2	2 060 055 578 301	88 359 230 989	60 166 511 652	24 461 154 321	3 731 565 017	68.1	6.25	2.37	1.832	37.28	0.114	0.762	0.943	1.058
x264_seek	1 966 389 483 167	85 688 434 921	58 863 030 064	23 684 590 688	3 140 814 169	68.7	6.41	2.41	1.873	32.75	0.124	0.858	1.005	1.080
gcc_pp_O3	224 853 966 645	17 984 502 435	877 592 244	16 879 105 802	227 804 388	4.9	1.23	15.01	8.283	43.74	0.855	4.688	0.551	1.420
gcc_pp_O2	266 929 823 668	21 831 478 671	1 077 827 255	20 499 147 410	254 504 006	4.9	1.25	15.36	8.497	39.97	0.807	4.473	0.548	1.349
gcc_smaller	254 671 373 628	17 265 308 635	931 220 362	16 129 309 979	204 778 295	5.4	1.09	12.67	10.106	49.17	1.456	3.248	0.497	1.186
gcc_ref32_O5	214 010 333 381	16 552 733 395	576 497 719	15 646 044 451	330 191 225	3.5	0.91	14.62	8.286	53.60	1.155	3.627	0.396	0.918
gcc_ref32_O3	318 165 395 272	17 589 262 663	566 719 818	16 818 061 154	204 481 691	3.2	0.68	10.57	10.394	74.28	1.973	5.027	0.508	1.271
mcf	888 079 703 023	7 407 299 183	3 229 620 454	4 144 174 494	33 504 235	43.6	1.40	0.93	21.442	74.48	5.221	6.187	2.463	5.125
perlbench_checkspam	1 309 521 527 045	120 008 085 121	23 137 601 197	93 974 994 957	2 895 488 967	19.3	7.98	14.35	1.958	34.15	0.091	0.390	0.167	0.366
perlbench_diffmail	778 566 201 895	66 166 562 175	3 721 969 520	61 702 382 195	742 210 461	5.6	1.96	15.85	4.308	39.09	0.119	6.097	0.125	0.773
perlbench_splitmail	736 991 491 190	79 002 095 413	7 227 608 106	70 427 685 127	1 346 802 180	9.1	4.16	19.11	8.748	8.56	0.029	6.699	0.027	1.134
bwaves_1	497 063 723 317	2 989 609 538	2 989 191 752	351 316	66 470	100.0	2.51	0.00	24.635	34.83	6.549	0.072	3.928	5.102
bwaves_2	784 691 829 793	4 892 060 128	4 891 557 364	404 109	98 655	100.0	2.59	0.00	24.250	35.39	6.276	0.084	6.900	4.895
bwaves_3	611 703 570 107	3 688 669 131	3 688 117 546	439 481	112 104	100.0	2.51	0.00	21.041	40.82	10.956	0.001	5.211	4.519
bwaves_4	745 048 195 941	4 606 093 801	4 605 716 550	319 055	58 197	100.0	2.56	0.00	19.674	43.40	10.477	0.001	4.118	5.478
exchange2	2 120 109 818 190	67 506 596 157	51 688 048 432	8 987 153 068	6 831 394 657	76.6	5.23	0.85	0.965	1.61	1.139	1.802	2.690	1.882
fotonik3d	2 457 798 604 788	19 150 475 290	18 811 665 225	309 878 797	28 931 268	98.2	3.36	0.03	8.473	81.02	2.732	0.027	0.018	3.311
roms	2 733 593 366 577	17 773 282 831	16 141 154 959	967 833 899	664 293 973	90.8	4.37	0.07	11.983	67.67	1.778	0.607	2.832	2.930
wrf	4 131 413 910 638	19 044 552 577	5 282 083 873	12 615 402 211	1 147 066 493	27.7	0.91	0.61	6.514	57.32	5.807	4.038	0.440	1.378
cam4	2 349 180 609 851	44 573 582 631	21 602 368 027	21 690 037 915	1 281 176 689	48.5	3.14	1.85	10.428	56.87	1.791	2.135	0.645	1.659
Total	58 391 814 127 468	1 959 776 631 856	**767 822 470 568**	1 084 245 704 359	107 708 456 932	39.2	3.78	3.71	0.000	—	—	—	—	—